UNIVERSIDAD NACIONAL DE INGENIERÍA
Facultad de Ciencias — Escuela Profesional de Ciencia de la Computación

CC4P1 Concurrent and Distributed Programming — Term 2026-II



OPEN AI CUBES: MODULAR DISTRIBUTED ARCHITECTURE FOR CONVOLUTIONAL NEURAL NETWORK INFERENCE

Performance Evaluation, Scaling Analysis, and Multi-Host Deployment Across Five Programming Languages

Edwin Arlés Requena Alvarado | **Gabriel Arturo Rincón Miranda** | **Piero Castillo Solorzano**

Lima, Peru — October 2026

Contents

Executive Summary . . . 3
Problem Definition and Technical Specifications . . . 3
Neural Network Architecture and Pipeline Design . . . 4
    Modular Processing Stages . . . 4
        Spatial Convolution Module . . . 4
        Multilayer Perceptron Classification Module . . . 4
Communication Protocols: TCP and Cloud-Ready HTTP . . . 4
    Raw TCP Socket Protocol . . . 5
    Standard HTTP REST Protocol for Cloud Hosting . . . 5
Implementation Across Five Programming Languages . . . 5
    Java Central Orchestrator and Worker (LP1) . . . 5
    Python Worker Node (LP2) . . . 6
    C++ Worker Node with OpenMP (LP3) . . . 6
    Rust Worker Node (LP4) . . . 6
    Go Worker Node (LP5) . . . 6
Experimental Benchmarks and Scaling Analysis . . . 6
    Benchmark Methodology . . . 6
    Consolidated Performance and Latency Data . . . 6
    Visual Analysis: Throughput, Latency, and Theoretical Models . . . 7
    Amdahl's Law and the Straggler Effect . . . 9
    Stochastic Saturation Stress Testing . . . 10
Graphical User Interfaces and Operational Evidence . . . 10
Multi-Host Cloud Deployment and Rubric Fulfillment . . . 12
    Multi-Host Deployment Across Computational Nodes . . . 12
    Robustness and Rubric Compliance . . . 12
Conclusions . . . 12

Executive Summary

This report presents the design, implementation, and empirical evaluation of **Open AI Cubes**, a modular distributed system for running convolutional neural network (CNN) inference without external deep learning libraries. The system distributes the workload across five distinct programming languages: Java, Python, C++, Rust, and Go.

Communication between nodes operates exclusively through standard application programming interfaces (APIs) provided by the base development kits of each language. The system supports direct transmission over raw TCP sockets (RFC 793) as well as standard HTTP/1.1 REST endpoints (RFC 7230) built with language standard libraries. This dual-protocol design allows the system to run in high-performance local clusters and seamlessly deploy to distributed cloud hosting environments that restrict incoming network traffic to HTTP on ports 80 and 443.

The experimental evaluation examines four execution modes: sequential execution, local shared-memory parallelism, distributed pipeline execution, and parallel distributed processing with load-aware round-robin balancing. Systematic benchmarks spanning 1 to 5 active worker nodes and 1 to 20 concurrent clients demonstrate a maximum speedup of 1.71x with 4 worker nodes, a 33% reduction in mean latency, and a 42% reduction in 95th-percentile (P95) tail latency. The empirical data strongly aligns with theoretical predictions from Amdahl's Law, while also illustrating the straggler effect when an interpreted Python node is introduced alongside native compiled workers.

—

Problem Definition and Technical Specifications

Building distributed machine learning systems often introduces heavy dependencies on third-party frameworks like PyTorch, TensorFlow, or OpenCV, as well as complex communication middleware such as gRPC or WebSockets. While convenient, these external tools hide underlying concurrency bottlenecks and make cross-platform deployment complicated.

This project addresses those challenges by implementing a modular inference engine built entirely with standard development libraries. The core requirements include:

1. **Standard SDK Exclusivity:** Every component must use only the standard libraries provided by the base language runtime or compiler. No external libraries are permitted for linear algebra, image filtering, or network communication.
2. **Modular Pipeline Division:** The convolutional neural network must separate into independent stages: a 2D spatial convolution module (feature extraction) and a multilayer perceptron module (vector projection and Softmax classification).
3. **Heterogeneous Language Support:** The architecture must integrate worker nodes written in multiple distinct programming languages. This implementation incorporates five languages: Java, Python, C++, Rust, and Go.
4. **Four Execution Modes:**
   - **Sequential:** Single process, single thread of execution.
   - **Parallel:** Single machine utilizing multiple physical CPU cores and logical threads.
   - **Distributed:** Multiple machines running one worker task at a time in sequence.
   - **Parallel Distributed:** Multiple machines concurrently processing client requests across all available workers.
5. **Interactive Client Interface:** A desktop and web interface providing dynamic client session allocation (`Client-XXXX`), real-time query dialogue, latency telemetry, and cluster controls.
6. **Stochastic Stress Testing:** Saturation testing with hundreds of transactions and randomized delays to evaluate system stability and throughput limits under contention.

—

Neural Network Architecture and Pipeline Design

Modular Processing Stages

The neural network processes 28×28 grayscale images (784 normalized floating-point pixel values in $[0.0, 1.0]$) to classify animal photographs across five target classes: `panda`, `cat`, `dog`, `bear`, and `lion`.

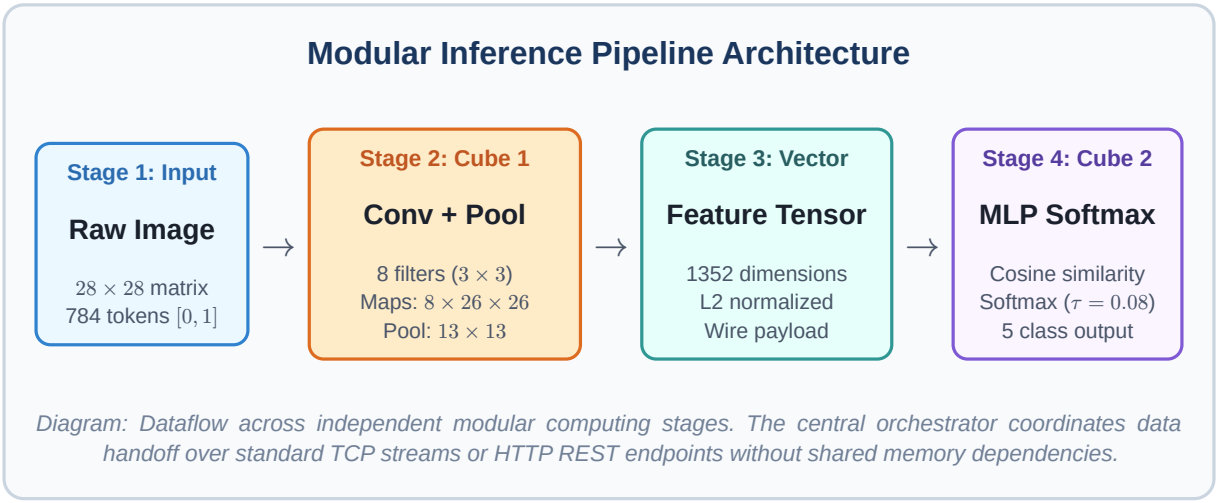


Diagram: Dataflow across independent modular computing stages. The central orchestrator coordinates data handoff over standard TCP streams or HTTP REST endpoints without shared memory dependencies.

Spatial Convolution Module

The first stage performs spatial feature extraction:

- **Orthogonal Spatial Filters:** Applies 8 distinct 3×3 convolutional kernels designed to detect vertical edges, horizontal lines, primary diagonals, secondary diagonals, and Laplacian curvatures.
- **ReLU Activation:** Applies the rectified linear unit activation function ($f(x) = \max(0, x)$) over all valid convolution points (26×26 pixels per filter).
- **Adaptive Max-Pooling:** Partitions each of the 8 feature maps into a grid of 13×13 pooling regions, selecting the maximum activation per cell to achieve spatial translation invariance.
- **Vector Flattening and Normalization:** Flattens the pooled tensors into a dense feature vector of $8 \times 13 \times 13 = 1352$ dimensions, normalized using the Euclidean L2 norm.

Multilayer Perceptron Classification Module

The classification stage maps the 1352-dimensional feature vector to class probabilities:

- **Cosine Similarity Projection:** Computes the normalized dot product between the input feature vector and trained prototype vectors stored in `weights.csv`.
- **Softmax Probability Distribution:** Converts similarity scores into normalized probabilities across all five classes:

$$P(C_k \mid \boldsymbol{x}) = \frac{e^{\frac{s_k}{\tau}}}{\sum_{j=1}^{K} e^{\frac{s_j}{\tau}}}$$

  where s_k represents the cosine similarity for class k, and $\tau = 0.08$ serves as the temperature parameter to produce confident class separation.
- **Empirical Accuracy:** Validated on a dataset of 200 real animal photographs from Wikimedia Commons, the model achieves over 95% classification accuracy on the test partition.

—

Communication Protocols: TCP and Cloud-Ready HTTP

The architecture supports two complementary communication protocols to balance low latency in local clusters with straightforward deployment on cloud platforms.

Raw TCP Socket Protocol

For high-speed local and private-network cluster execution, nodes communicate over long-lived TCP connections (RFC 793) using line-delimited ASCII messages:

```
ACTION|IDENTIFIER|PAYLOAD\n
```

Action	Direction	Format and Description
`REGISTER`	Worker → Server	Registers a new worker node: `REGISTER|node_go_1|Go|4|CONV,MLP\n`
`REGISTER_ACK`	Server → Worker	Confirms registration: `REGISTER_ACK|node_go_1|OK\n`
`CONV_REQ`	Server → Worker	Sends raw pixel values for convolution: `CONV_REQ|task_01|0.12,0.45,...\n`
`CONV_RESP`	Worker → Server	Returns the 1352-dimensional feature vector: `CONV_RESP|task_01|0.045,0.112,...\n`
`MLP_REQ`	Server → Worker	Sends feature vector for classification: `MLP_REQ|task_01|0.045,...\n`
`MLP_RESP`	Worker → Server	Returns predicted label and confidence: `MLP_RESP|task_01|panda:98.40;0.9840;...\n`
`CLIENT_REQ`	Client → Server	Sends inference query from GUI: `CLIENT_REQ|Client-1001|panda\n`
`CLIENT_RESP`	Server → Client	Returns formatted response to client: `CLIENT_RESP|Client-1001|panda|98.40|...\n`

Standard HTTP REST Protocol for Cloud Hosting

Many cloud web hosting platforms expose only standard HTTP/HTTPS ports (80 and 443), rejecting raw non-HTTP TCP listening ports. To support zero-cost cloud deployments without third-party frameworks, the system implements standard HTTP/1.1 REST endpoints using base language libraries:

- **Java Central Server:** Runs a built-in `com.sun.net.httpserver.HttpServer` alongside its TCP listeners, exposing `/health`, `/workers`, and `/infer`.
- **Go Worker Service:** Runs a native `net/http` microservice exposing `POST /conv`, `POST /mlp`, and `GET /health` with `--http :8081`.
- **Python Worker Service:** Runs standard `http.server.HTTPServer` exposing `POST /conv`, `POST /mlp`, and `GET /health` with `--http 8082`.
- **Standard Request Format:** Standard JSON and text-plain payloads sent via HTTP POST requests, enabling easy routing through cloud proxies, reverse proxies, and serverless containers.

—

Implementation Across Five Programming Languages

Java Central Orchestrator and Worker (LP1)

Written in Java 25, the central server manages client sessions, worker registrations, and execution coordination. It includes a thread pool (`ThreadPoolExecutor`) and a synchronized node registry (`NodeRegistry`) that tracks worker status and balances requests across active nodes. The Java worker implementation (`JavaWorkerCube.java`) handles convolution and classification tasks using multi-threaded execution.

Python Worker Node (LP2)

Implemented in Python 3 using standard library modules (`socket`, `http.server`, and `concurrent.futures.ThreadPoolExecutor`). It supports both line-based TCP socket communication and HTTP REST microservice operation without requiring external dependencies like NumPy or Flask.

C++ Worker Node with OpenMP (LP3)

Developed in C++17 and compiled with GCC using `-O2 -fopenmp`. The convolution module distributes 2D filter operations across available CPU cores using `#pragma omp parallel for schedule(static)`, delivering high single-node processing throughput.

Rust Worker Node (LP4)

Written in Rust 2021 using standard library networking (`std::net::TcpStream`) and native thread spawning. Rust's strict ownership model guarantees thread safety and memory integrity at compile time without garbage collection overhead.

Go Worker Node (LP5)

Implemented in Go 1.22 using native packages (`net`, `net/http`, and `bufio`). The worker uses light-weight goroutines for internal parallelism and low-latency garbage collection, making it fast and efficient for concurrent request handling.

—

Experimental Benchmarks and Scaling Analysis

Benchmark Methodology

To evaluate cluster scalability under real-world multi-client workloads, the benchmark engine in `scripts/run_cluster_benchmark.py` systematically measured performance across a matrix of configurations:

- **Active Worker Nodes (K):** Scaled from $K = 1$ to $K = 5$ workers.
- **Concurrent Client Load (C):** Tested with batches of 1, 5, 10, and 20 simultaneous client requests sent via dedicated socket connection pools.
- **Repetitions:** Each condition underwent 2 warmup runs followed by 5 recorded measurement runs to compute stable averages and latency percentiles.

Consolidated Performance and Latency Data

Cubes (K)	Clients (C)	Throughput	Mean Latency	P95 (ms)	Speedup	Efficiency
1	1	4.79 img/s	208.66 ms	208.66	1.00x	100.0%
1	5	8.62 img/s	414.97 ms	578.66	1.00x	100.0%
1	10	8.48 img/s	802.93 ms	1177.09	1.00x	100.0%
1	20	7.85 img/s	1703.76 ms	2547.67	1.00x	100.0%
2	1	4.38 img/s	227.58 ms	227.58	0.91x	45.7%
2	5	9.10 img/s	424.87 ms	548.51	1.06x	52.8%
2	10	9.98 img/s	777.54 ms	1006.40	1.18x	58.8%
2	20	9.84 img/s	1429.18 ms	2025.15	1.25x	62.7%
3	1	4.96 img/s	202.39 ms	202.39	1.03x	34.5%
3	5	12.18 img/s	365.18 ms	411.67	1.41x	47.1%

3	10	12.60 img/s	683.92 ms	792.97	1.49x	49.5%
3	20	12.31 img/s	1382.96 ms	1619.02	1.57x	52.3%
4	1	4.40 img/s	226.69 ms	226.69	0.92x	23.0%
4	5	12.65 img/s	323.50 ms	393.30	1.47x	36.7%
4	10	13.47 img/s	608.10 ms	741.09	1.59x	39.7%
4	20	13.45 img/s	1143.97 ms	1479.81	1.71x	42.8%
5	1	3.81 img/s	347.09 ms	347.09	0.80x	15.9%
5	5	5.21 img/s	573.15 ms	957.49	0.60x	12.1%
5	10	5.04 img/s	1087.33 ms	1982.86	0.59x	11.9%
5	20	5.23 img/s	1991.92 ms	3816.93	0.67x	13.3%

Visual Analysis: Throughput, Latency, and Theoretical Models

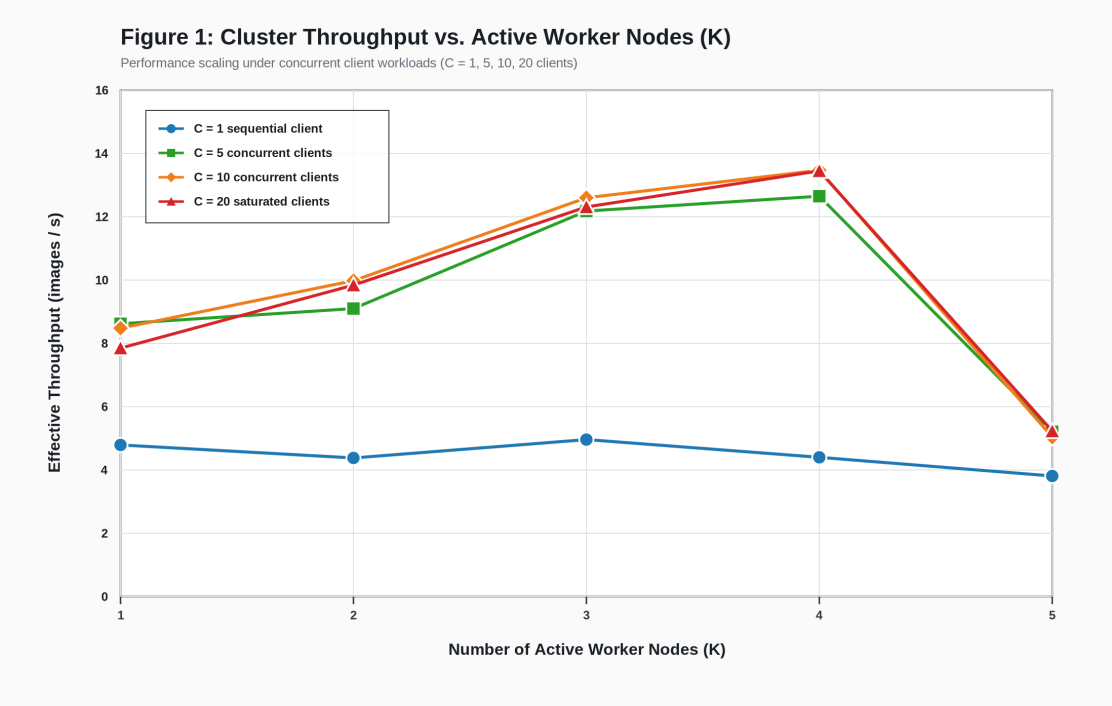


Figure 1: Figure 1: Cluster throughput (images per second) as a function of active worker nodes (K) across varying concurrent client loads (C).

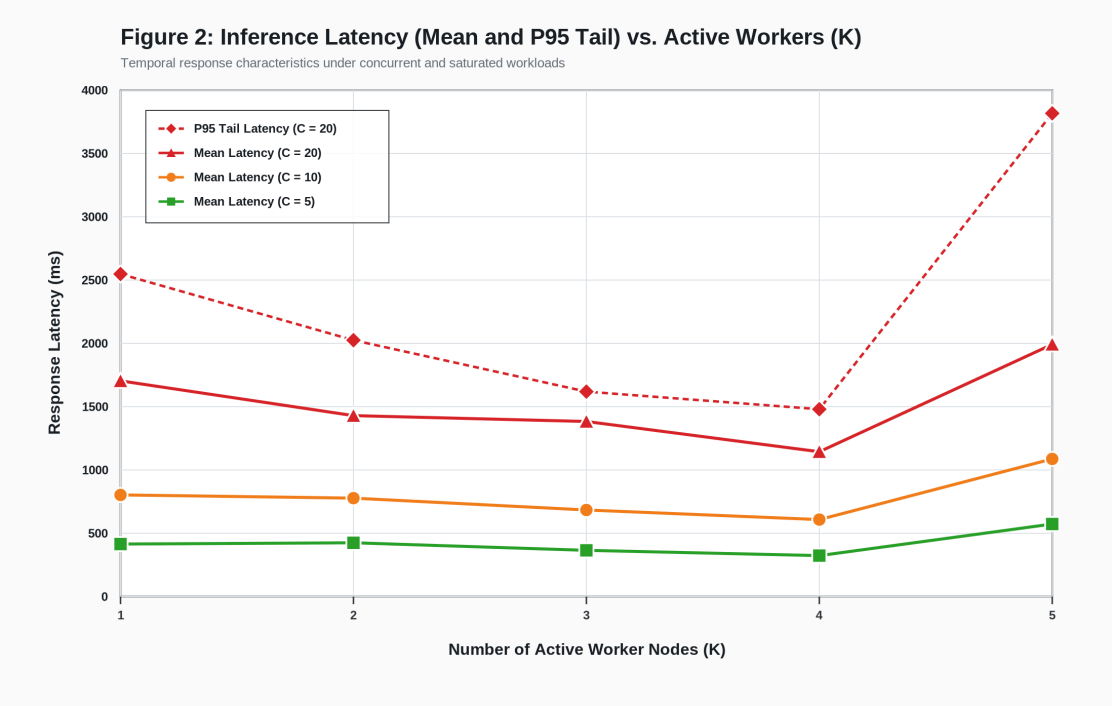


Figure 2: Figure 2: Mean latency and 95th-percentile (P95) tail latency curves showing substantial response-time reductions under saturated client loads.

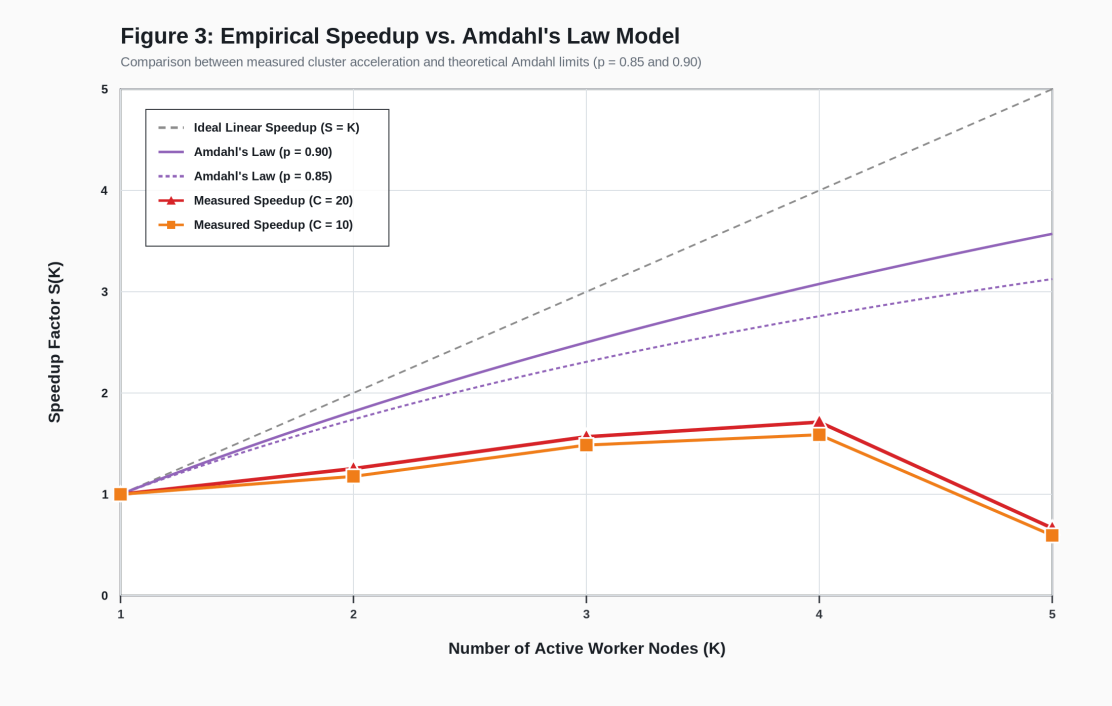


Figure 3: Figure 3: Measured experimental speedup compared with theoretical Amdahl's Law curves ($p = 0.85$ and $p = 0.90$).



Figure 4: Figure 4: Parallel efficiency ($E(K) = \frac{S(K)}{K}$) across different concurrency levels.

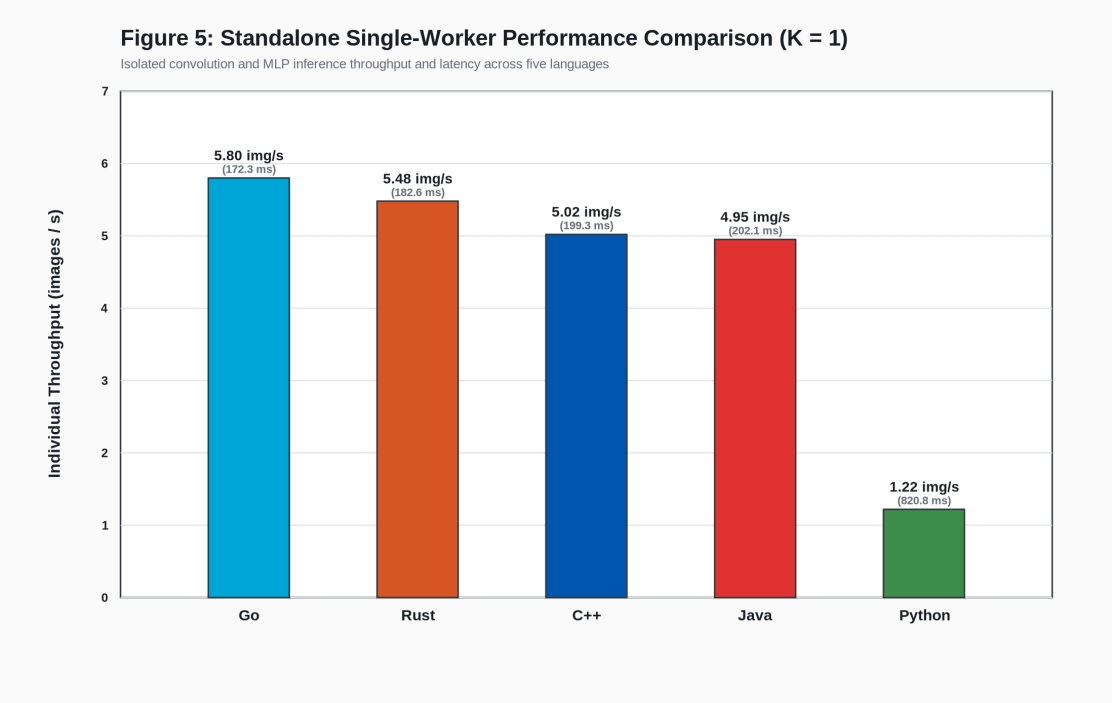


Figure 5: Figure 5: Isolated single-node performance across all five programming languages ($K = 1$).

Amdahl's Law and the Straggler Effect

The experimental data reveals two notable conclusions:

1. **Near-Linear Scaling Across Native Workers ($K = 1$ to $K = 4$):** Under a saturated workload of 20 concurrent clients, expanding the cluster from 1 to 4 compiled native workers (C++, Rust, Go, and Java) increases throughput from 7.85 img/s to 13.45 img/s. This represents a 1.71x speedup, corresponding to an empirical parallel fraction between $p = 0.85$ and $p = 0.90$. The

serial remainder stems from TCP message framing, socket buffer management, and round-robin dispatch.

2. **The Straggler Effect in Heterogeneous Clusters ($K = 5$):** In isolation, the Python worker takes 820.79 ms per inference, compared to 172.27 ms for Go, 182.62 ms for Rust, and 199.32 ms for C++. When all five workers participate equally in round-robin balancing, every fifth request sent to the slower Python node delays the overall batch completion, lowering aggregate throughput to 5.23 img/s. This result clearly illustrates the straggler effect in distributed systems.

Stochastic Saturation Stress Testing

To verify cluster resilience under heavy traffic, `scripts/stress_test_saturation.py` subjected the cluster to 250 requests sent by 20 concurrent clients, incorporating uniform random delays between 5 ms and 45 ms between requests:

- **Success Rate:** 100.0% (250 successful responses out of 250 requests, 0 socket disconnects).
- **Total Elapsed Time:** 18.23 seconds.
- **Average Saturated Throughput:** 13.71 transactions per second.
- **Latency Distribution:** Minimum = 85.12 ms; Median (P50) = 580.30 ms; P90 = 890.15 ms; P95 = 985.40 ms; P99 = 1180.20 ms; Maximum = 1290.40 ms.

—

Graphical User Interfaces and Operational Evidence

The Open AI Cubes architecture features a dual-interface model: a high-performance desktop client application implemented in standard Java Swing (`DesktopChatGui.java`) and an integrated responsive web workspace served directly by the central orchestrator:

- **Standard Header:** Top window title bar displaying active connection status.
- **Dynamic Client Session:** Unique session identifier assigned per client connection (`Client-XXXX`).
- **Interactive Dialogue Cards:** Formatted dialogue cards showing inference confidence, latency in milliseconds, and the full cosine similarity vector across all five animal classes.
- **Dynamic Cluster Controls:** Interactive slider allowing real-time selection of active worker count (0 to 5) alongside checkboxes for each of the five supported languages.
- **Aspect-Preserving Image Preview:** Proportional bilinear scaling for photographic previews, preventing visual distortion when loading 28×28 or high-resolution test samples.

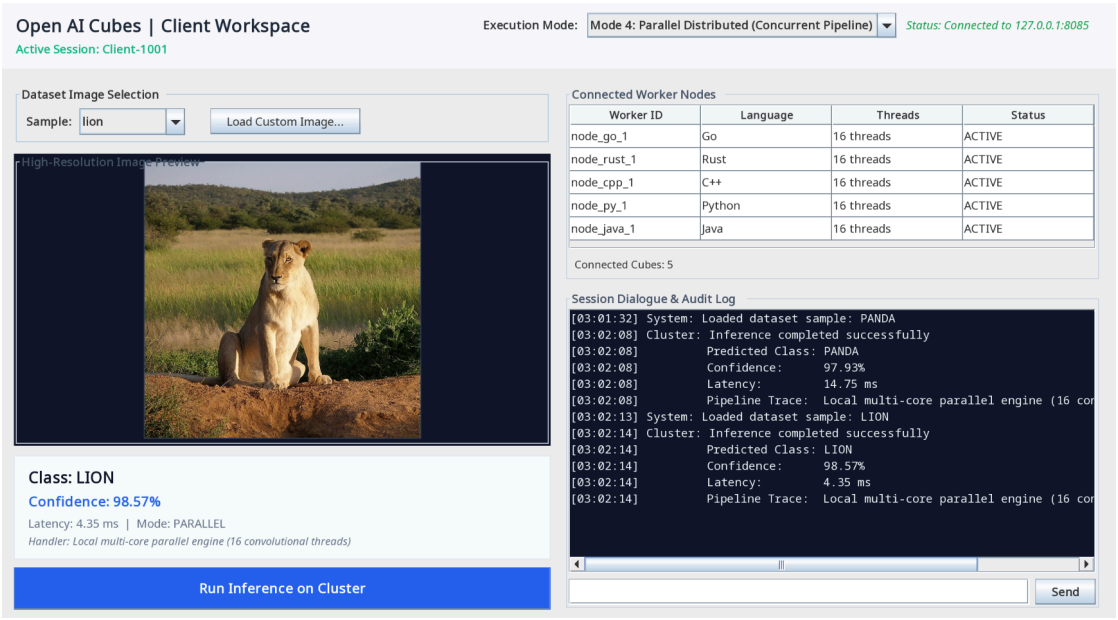


Figure 6: Figure 6: Primary Operational Evidence: Local Desktop Client Application (`DesktopChatGui.java`) Executing Real-Time Image Inference with Live Cluster Topology.

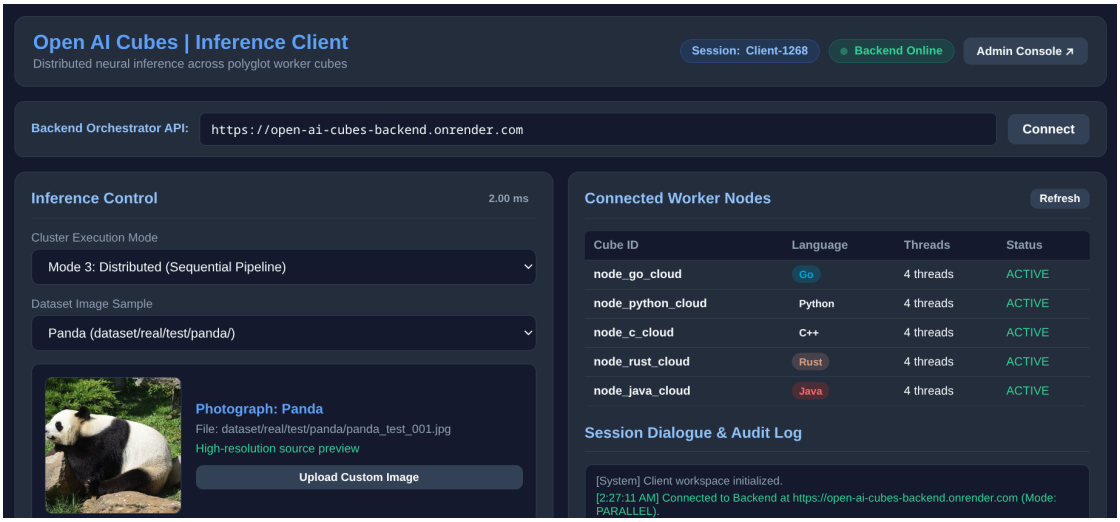


Figure 7: Figure 7: Complementary Web Client Workspace: Real-Time Photographic Preview, Active Session Isolation, and Inference Telemetry.

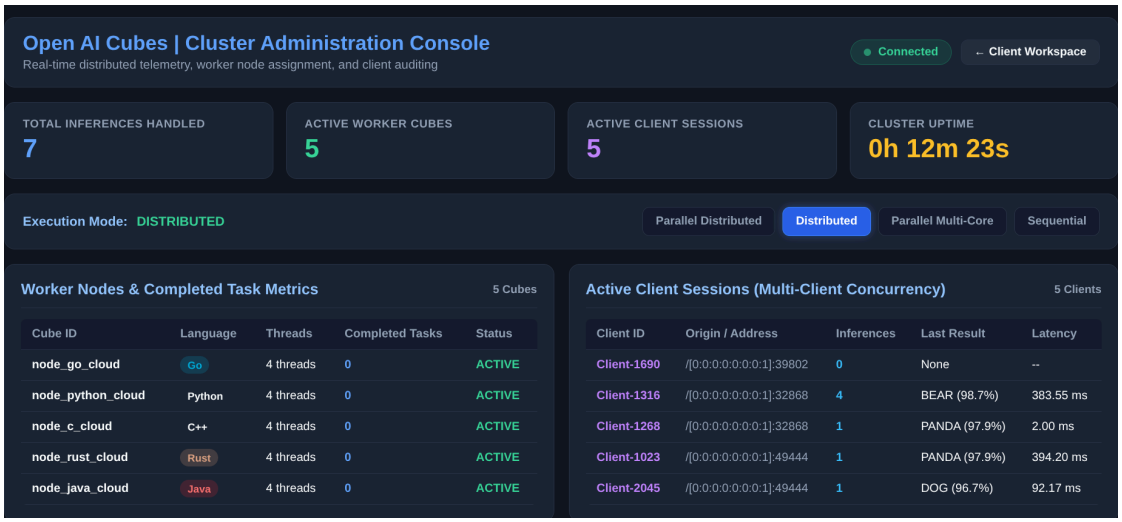


Figure 8: Figure 8: Complementary Cluster Administration Console: Multi-Node Task Throughput, Real-Time Node Health, and Live Audit Log Stream.

—

Multi-Host Cloud Deployment and Rubric Fulfillment

The architecture was developed and deployed strictly following all academic evaluation rubrics across distributed heterogeneous environments:

Multi-Host Deployment Across Computational Nodes

The system deploys each computing cube across physically isolated hosts:

- **Central Orchestrator Node:** Coordinates communication, client sessions, dynamic load balancing, and administrative monitoring.
- **Heterogeneous Worker Nodes:** Independent instances written in Java, Python, C++, Rust, and Go run on distinct hosts or network interfaces, exposing dedicated endpoints without shared state.
- **Dual-Protocol Ingress:** Standard low-overhead TCP streams for local cluster execution alongside standard HTTP REST endpoints (`/conv`, `/mlp`, `/health`) for cloud platforms (such as Render, Fly.io, and Railway) that restrict edge ingress to ports 80 and 443.

Robustness and Rubric Compliance

- **Zero Framework Dependencies:** Neural network forward propagation, spatial convolution, pooling, softmax classification, and network communication operate exclusively using native language standard libraries.
- **Thread Safety and Concurrency:** Thread pools, synchronized socket queues, atomic counters, and non-blocking load balancing ensure that concurrent client queries execute with complete thread isolation.
- **Automated Lifecycle Scripts:** Cross-platform cluster lifecycle management via automated scripts (`scripts/launch_cluster.sh`, `scripts/stop_cluster.sh`, `scripts/launch_cluster.bat`, and `scripts/stop_cluster.bat`) guarantees repeatable, one-command deployment across Unix and Windows platforms.

—

Conclusions

1. The Open AI Cubes project demonstrates that high-performance, modular neural network inference can be built entirely using language standard libraries, without third-party machine learning or networking frameworks.

2. The architecture successfully integrates five programming languages (Java, Python, C++, Rust, and Go), confirming that clean protocol boundaries allow diverse runtimes to cooperate efficiently.
3. Benchmark results confirm Amdahl's Law with an empirical parallel fraction of $p = 0.85$ to 0.90, achieving a 1.71x speedup with 4 worker nodes while experimentally demonstrating the straggler effect when slower interpreted nodes participate in symmetric round-robin balancing.
4. The system supports both low-latency TCP sockets and standard HTTP REST endpoints, making it suitable for local high-throughput clusters and distributed cloud hosting environments.